

Architecture Decision Records (ADRs)

ADR 01: Selection of Frameworks & Runtime Environment

Context -

The contract analysis prototype requires a highly responsive frontend for document uploading and interactive analysis, paired with a high-performance backend capable of handling asynchronous text processing, file streaming, and heavy AI orchestration under tight latency constraints.

Decision -

- **Frontend:** React.js (Single Page Application architecture).
- **Backend:** Python 3.10+ with FastAPI.

Status – Accepted

Consequences

- **FastAPI Asynchronous Engine:** FastAPI utilizes an ASGI (Asynchronous Server Gateway Interface) backend built on uvicorn and pydantic. This allows the server to process concurrent file upload streams and long-running HTTP network requests to external LLM providers without blocking the main event loop.
- **Component-Driven UI Reactivity:** React's virtual DOM ensures real-time UI state switching (e.g., toggling loading screens, rendering incremental text changes via react-markdown) without requiring page updates, heavily improving user experience.
- **Loose Coupling via REST:** Splitting the stack into a React UI and a FastAPI REST layer ensures the system remains highly modular. The entire user interface can be refactored or ported to mobile in the future without touching the underlying core RAG/Agent business logic.

ADR 02: Retrieval-Augmented Generation (RAG) Architecture & Vector Store Selection

Context

Legal contracts contain sensitive, proprietary business logic, indemnification obligations, and liability boundaries. The application must accurately extract context from multiple legal agreements concurrently and present it to the LLM within strict token budgets, without exposing full documents unnecessarily to third-party databases.

Decision

- **RAG Framework:** LangChain.
- **Vector Database:** ChromaDB (Local Embedded Mode).
- **Embedding Model:** HuggingFace all-MiniLM-L6-v2 (Local Execution).
- **Text Splitter:** RecursiveCharacterTextSplitter (Chunk Size: 1000 tokens, Overlap: 200 tokens).

Status – Accepted

Consequences

- **Data Isolation and Privacy:** By leveraging an embedded instance of **ChromaDB**, all vectorized contract text chunks are kept strictly on the local file system (chroma_data/). No contract metadata or text data is leaked to cloud-hosted vector engines, fulfilling strict data isolation requirements.
- **Semantic Chunking Strategy:** Legal documents contain multi-layered nested clauses. Standard character splitting destroys semantic context. Using RecursiveCharacterTextSplitter ensures that paragraphs, sections, and list elements are broken down gracefully, preserving exact structural meaning.
- **Deterministic Retrieval:** Configuring similarity search parameters at k=20 or k=25 ensures that relevant paragraphs across multiple distinct uploaded contracts are pulled into context simultaneously, enabling accurate cross-contract clause comparison.

ADR 03: AI Agent Strategy & Structured Output Orchestration

Context

The system must execute diverse legal workflows (Summarise, Identify Risk, Extract Obligations, Compare Clauses). Standard text-based LLM completion is non-deterministic, highly volatile, and frequently outputs formatting variations (such as raw text walls or improper XML tags) that crash UI components.

Decision

- **Agent Paradigm:** Structured Output Routings via Pydantic Data Verification.
- **Inference Engine:** Groq Cloud API running **LLaMA 3 70B**.

Status – Accepted

Consequences

- **Guaranteed Schema Enforcement:** We coupled LangChain's `.with_structured_output()` syntax directly with a rigid Pydantic AgentOutput model, enforcing a strict JSON schema: `{"per_contract_breakdown": "string", "reasoning": "string"}`. This forces LLaMA 3 to respond strictly within an explicit payload. If the model strays, the framework throws a validation error before it hits the UI, preventing frontend crashes.
- **Unmatched Inference Speed:** Legal documents demand massive context windows. Utilizing Groq's LPU (Language Processing Unit) architecture allows the agent to ingest thousands of context tokens and stream back fully verified JSON schemas in seconds, rendering instantaneous UI results.
- **Mandatory Explainability:** By hardcoding a specific reasoning field directly into the core Pydantic structure, the agent is forced to process its internal logic step-by-step alongside the data, satisfying the assignment's explicit requirement for visible agent reasoning.

ADR 04: Component UI Rendering and Markdown Parsing Strategy

Context

The AI Agent generates complex, multi-layered text structures (such as bulleted lists, bolded headers, and distinct section breaks) wrapped inside single string JSON fields. Standard React string binding renders raw text blocks without formatting, while dangerously parsing raw HTML creates massive Cross-Site Scripting (XSS) vulnerability vectors.

Decision

- **Library:** Implement the native react-markdown library inside the primary UI view wrapper.
- **Styling:** Enforce deterministic styling overrides via decoupled App.css cascade rules.

Status – Accepted

Consequences

- **Safe Semantic Parsing:** react-markdown securely tokenizes incoming strings into standard HTML abstract syntax trees without utilizing risky JavaScript innerHTML configurations. This guarantees that Markdown characters effortlessly transition into standard semantic layout containers.
- **Visual Density Reduction:** By explicitly configuring spacing metrics within the stylesheet (e.g., setting line heights to 1.6 and bottom margins to 10px), we successfully converted dense blocks of text into readable, scannable structures.
- **Prompt-Level Isolation:** Rather than stripping formatting using client-side processing utilities, formatting rules are handled at the LLM system-prompt level. This ensures that only true data headers are emphasized, preserving clean layout boundaries across all agent reports.